

User Manual

for S32 I2C Driver

Document Number: UM11I2CASR4.4 Rev0000R4.0.2 Rev. 1.0

1 Revision History	2
2 Introduction	3
2.1 Supported Derivatives	3
2.2 Overview	4
2.3 About This Manual	4
2.4 Acronyms and Definitions	5
2.5 Reference List	5
3 Driver	6
3.1 Requirements	6
3.2 Driver Design Summary	6
3.3 Hardware Resources	7
3.4 Deviations from Requirements	7
3.5 Driver Limitations	7
3.6 Driver usage and configuration tips	8
3.7 Runtime errors	11
3.8 Symbolic Names Disclaimer	12
4 Tresos Configuration Plug-in	13
4.1 Module I2c	14
4.2 Container GeneralConfiguration	15
4.3 Parameter I2cDevErrorDetect	15
4.4 Parameter I2cTimeoutMethod	16
4.5 Parameter I2cDmaUsed	16
4.6 Parameter I2cDmaTransferErrorDetect	17
4.7 Parameter I2cDisableDemReportErrorStatus	17
4.8 Parameter I2cMulticoreSupport	18
4.9 Parameter I2cEnableUserModeSupport	18
4.10 Parameter I2cTimeoutDuration	20
4.11 Parameter I2cVersionInfoApi	20
4.12 Parameter I2c_Callback	21
4.13 Parameter I2cCallback	21
4.14 Parameter I2c_ErrorCallback	22
4.15 Parameter I2cErrorCallback	22
4.16 Container I2cGlobalConfig	23
4.17 Reference I2cEcucPartitionRef	23
4.18 Container I2cChannel	24
4.19 Parameter I2cChannelId	24
4.20 Parameter I2cHwChannel	25
4.21 Parameter I2cMasterSlaveConfiguration	25

4.22 Reference I2cChannelEcucPartitionRef	26
4.23 Container I2cMasterConfiguration	26
4.24 Parameter I2cAsyncMethod	27
4.25 Parameter I2cMasterGlitchFilter	27
4.26 Parameter I2cIbcValue	28
4.27 Parameter I2cSCLDivider	28
4.28 Parameter I2cSDAHoldDelay	29
4.29 Parameter I2cSCLHoldStart	30
4.30 Parameter I2cSCLHoldStop	30
4.31 Parameter I2cBaudRate	31
4.32 Reference I2cClockRef	31
4.33 Reference I2cTxDmaChannel	32
4.34 Reference I2cRxDmaChannel	32
4.35 Container I2cSlaveConfiguration	33
4.36 Parameter I2cSlaveAddress	33
4.37 Parameter I2cSlaveListening	34
4.38 Parameter I2cSlaveGlitchFilter	34
4.39 Container I2cDemEventParameterRefs	35
4.40 Reference I2C_E_TIMEOUT_FAILURE	35
4.41 Container CommonPublishedInformation	36
4.42 Parameter ArReleaseMajorVersion	36
4.43 Parameter ArReleaseMinorVersion	37
4.44 Parameter ArReleaseRevisionVersion	37
4.45 Parameter ModuleId	38
4.46 Parameter SwMajorVersion	38
4.47 Parameter SwMinorVersion	39
4.48 Parameter SwPatchVersion	39
4.49 Parameter VendorApiInfix	40
4.50 Parameter VendorId	41
5 Module Index	42
5.1 Software Specification	42
6 Module Documentation	43
6.1 I2c Driver	43
6.1.1 Detailed Description	43
6.1.2 Data Structure Documentation	45
6.1.3 Macro Definition Documentation	49
6.1.4 Types Reference	50
6.1.5 Enum Reference	50
6.1.6 Function Reference	52

6.1.7 Variable Documentation	57
6.2 IIC Driver	58
6.2.1 Detailed Description	58
6.2.2 Data Structure Documentation	60
6.2.3 Macro Definition Documentation	61
6.2.4 Enum Reference	64
6.2.5 Function Reference	66



Chapter 1

Revision History

Revision	Date	Author	Description
1.0	30.06.2023	NXP RTD Team	S32 Real-Time Drivers AUTOSAR 4.4 Version 4.0.2 Release

Chapter 2

Introduction

- [Supported Derivatives](#)
- [Overview](#)
- [About This Manual](#)
- [Acronyms and Definitions](#)
- [Reference List](#)

This User Manual describes NXP Semiconductor I2C for S32. I2C driver configuration parameters and deviations from the specification are described in Driver chapter of this document. I2C driver requirements and APIs are described in the I2C driver software specification document.

2.1 Supported Derivatives

The software described in this document is intended to be used with the following microcontroller devices of NXP Semiconductors:

- s32g274a_bga525
- s32g254a_bga525
- s32g233a_bga525
- s32g234m_bga525
- s32g378a_bga525
- s32g379a_bga525
- s32g398a_bga525
- s32g399a_bga525
- s32g338m_bga525
- s32g339m_bga525
- s32g358a_bga525
- s32g359a_bga525
- s32r45_bga780

All of the above microcontroller devices are collectively named as S32.

2.2 Overview

AUTOSAR (AUTomotive Open System ARchitecture) is an industry partnership working to establish standards for software interfaces and software modules for automobile electronic control systems.

AUTOSAR:

- paves the way for innovative electronic systems that further improve performance, safety and environmental friendliness.
- is a strong global partnership that creates one common standard: "Cooperate on standards, compete on implementation".
- is a key enabling technology to manage the growing electrics/electronics complexity. It aims to be prepared for the upcoming technologies and to improve cost-efficiency without making any compromise with respect to quality.
- facilitates the exchange and update of software and hardware over the service life of the vehicle.

2.3 About This Manual

This Technical Reference employs the following typographical conventions:

- **Boldface** style: Used for important terms, notes and warnings.
- *Italic* style: Used for code snippets in the text. Note that C language modifiers such "const" or "volatile" are sometimes omitted to improve readability of the presented code.

Notes and warnings are shown as below:

Note

This is a note.

Warning

This is a warning

2.4 Acronyms and Definitions

Term	Definition
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
ASM	Assembler
BSW	Basic Software
DEM	Diagnostic Event Manager
DET	Development Error Tracer
C/CPP	C and C++ Source Code
ECU	Electronic Control Unit
I2C	Inter-Integrated Circuit
ISR	Interrupt Service Routine
N/A	Not Applicable
VLE	Variable Length Encoding

2.5 Reference List

#	Title	Version
1	S32G2 Reference Manual	Rev. 7, February 2023
2	S32G3 Reference Manual	Rev. 3, February 2023
3	S32R45 Reference Manual	Rev. 3, December 2021
4	S32G2: Mask Set Errata for Mask 0P77B	Rev. 3.0
5	S32G2: Mask Set Errata for Mask 1P77B	Rev. 1.0
6	S32G3: Mask Set Errata for Mask 1P72B	Rev. 2.1
7	S32R45: Mask Set Errata for Mask P57D	Rev. 2.0
8	S32G2 Data Sheet	Rev. 6, December 2022
9	S32G3 Data Sheet	Rev. 2, RC, February 2023
10	VR5510 Data Sheet	Rev. 5, April 2022
11	S32R45 Data Sheet	Rev. 3, RC — November 2022

Chapter 3

Driver

- [Requirements](#)
- [Driver Design Summary](#)
- [Hardware Resources](#)
- [Deviations from Requirements](#)
- [Driver Limitations](#)
- [Driver usage and configuration tips](#)
- [Runtime errors](#)
- [Symbolic Names Disclaimer](#)

3.1 Requirements

Requirements for this driver are detailed in the Autosar Driver Software Specification document (See [Table Reference List](#)).

For CDD: I2c is a Complex Device Driver (CDD), so there are no AUTOSAR requirements regarding this module.

It has vendor-specific requirements and implementation.

3.2 Driver Design Summary

The I2c driver is implemented as an Autosar complex device driver. It uses the I2c hardware peripheral which provides support for implementing the I2c protocol. The I2c driver implements both master and slave mode for I2c channels. The driver offers a hardware independent API to the upper layer that can be used to configure the I2c and initiate synchronous and asynchronous data transfers. Asynchronous transfer for a master channel uses interrupts. The slave operates only in asynchronous mode. Hardware and software settings can be configured using an Autosar standard configuration tool. The information required for an I2c data transfer will be configured in a data structure that will be sent as parameter to the API of the driver. The driver reports errors to the error manager as defined in AUTOSAR.

3.3 Hardware Resources

The hardware configured by the I2c driver is the same between derivatives.

Physical I2c Channels:

S32G2XX: IIC_0, IIC_1, IIC_2, IIC_3, IIC_4.

S32G3XX: IIC_0, IIC_1, IIC_2, IIC_3, IIC_4.

S32R45: IIC_0, IIC_1.

3.4 Deviations from Requirements

The driver deviates from the I2C Driver software specification in some places. The table below identifies the requirements that are not implemented or out of scope for the I2C Driver.

Term	Definition
N/S	Out of scope
N/I	Not implemented
N/F	Not fully implemented

Below table identifies the requirements that are not fully implemented, implemented differently or out of scope for the I2C driver.

Requirement	Status	Description	Notes
SWS_CDD_I2C_00047	N/S	The I2C module shall reset the interrupt flag at the end of the ISR (if not done automatically by hardware).	Replaced by CPR_RTD_00516

3.5 Driver Limitations

- For Master mode, the driver does not support:
 - 'b10BitsSlaveAddressSize = TRUE'
 - 'bExpectNack = TRUE'
 - 'bHighSpeedMode = TRUE' (No hardware support)
- For S32G3XX: I2C controller is unable to generate clocks when SDA is low Based on the I2C specification version 6.0, if the data line (SDA) is stuck low, the master should send nine clock pulses and the I2C slave device that holds the bus low should release it sometime within those nine clocks. But the I2C controller cannot generate nine clock pulses, as a result the slave device is unable to release the bus (recommend in Erratas ERR050686).

3.6 Driver usage and configuration tips

- Steps to configure I2c using DMA mode

1. Enable check "Enable Dma Support" from "General" tab of Mcl component.

The screenshot displays the configuration interface for the Mcl component. The 'General' tab is selected, showing the following settings:

- Name:** Mcl
- Post Build Variant Used:** ☒ (checked)
- Config Variant:** VariantPostBuild
- Mcl General Configuration:**
 - Name:** MclGeneral
 - Enable Production Error Reporting:** ☐ (unchecked)
 - Enable Development Error Detect:** ☐ (unchecked)
 - Mcl Enable User Mode Support:** ☒ (checked)
- Mcl Dma:**
 - Name:** MclDma
 - Enable DMA Support:** ☒ (checked)
 - Enable CRC Support:** ☐ (unchecked)
- Mcl Cache:**
 - Name:** MclCache
 - Enable CACHE Support:** ☒ (checked)
 - Enable CACHE Synchronization:** ☐ (unchecked)

2. Configure dma logic instance from "Dma logic instance" tab. Create two instances, one for transmit and one for receive transfers.

dmaLogicInstance_ConfigType

Name  dmaLogicInstance_ConfigType_0

Instance

Logic Instance Name  DMA_LOGIC_INST_0 

Hardware Instance  DMA_IP_HW_INST_0 

Debug  ☐  Round Robin Channel Arbitration  ☐ 

Halt After Error  ☐  Global Channel linking  ☐ 

Global Master ID Replication  ☐ 

Dma Crc

Name  dmaLogicInstance_DmaCrc

Enable Swap Bit  ☐ Enable Swap Byte  ☐

Enable Global  ☐

3. In "Logic Channel Configuration" an entry should be added for each logical instance. Choose hardware instance, hardware channel, set the corresponding "Interrupt Callback" and enable the "Enable Global Config" check.

The "Interrupt Callback" should have one of the following values: I2C0_DmaCompleteNotification, I2C1_DmaCompleteNotification, I2C2_DmaCompleteNotification, I2C3_DmaCompleteNotification, I2C4_DmaCompleteNotification, depending of the I2c instance that is used. For example, if the Dma channel is configured for I2C4, the value of Interrupt Callback should be I2C4_DmaCompleteNotification. This function is defined in I2c_Ip.c file and it is called at the end of the dma transfer.

Logic Channel

Name  I2C4_TX

Logic Channel Configuration Global Transfer ScatterGather Crc

Logic Channel Name  DMA_LOGIC_CH_0 

Hardware Instance  DMA_IP_HW_INST_0 

Hardware Channel  DMA_IP_HW_CH_17 

Interrupt Callback  I2C4_DmaCompleteNotification 

Error Interrupt Callback  NULL_PTR 

Enable Global Config  ☒  Enable Transfer Config  ☐ 

Enable Scatter/Gather  ☐  Enable CRC Config  ☐

Optional the user could implement the Error Interrupt Callback for the DMA channels used. This is useful in the development phase, when DMA errors may occur because of a faulty DMA configuration. In case a DMA error occurred, the I2c driver will return STATUS_I2C_IP_BUSY.

Logic Channel

Name*  I2C4_TX

Logic Channel Configuration Global Transfer ScatterGather Crc

Logic Channel Name  DMA_LOGIC_CH_0 

Hardware Instance  DMA_IP_HW_INST_0 

Hardware Channel*  DMA_IP_HW_CH_17 

Interrupt Callback*  I2C4_DmaCompleteNotification 

Error Interrupt Callback*  I2C4_DmaErrorNotification 

4. In "Global" tab the corresponding DMAMUX source should be selected. The "Enable DMAMUX Source" and "Enable DMA Request" checks should be enabled.

Logic Channel

Name  I2C4_TX

Logic Channel Configuration Global Transfer ScatterGather Crc

Enable Master Id Replication  ☐  Enable Buffered Writes  ☐ 

Request

Name  dmaLogicChannelConfig_GlobalRequestType

Enable DMAMUX Trigger  ☐  Enable DMAMUX Source  ☒ 

DMAMUX0 Source  DMA_IP_REQ_MUX0_DISABLED 

DMAMUX1 Source  DMA_IP_REQ_MUX1_I2C4_TX 

DMAMUX2 Source  DMA_IP_REQ_MUX2_DISABLED 

DMAMUX3 Source  DMA_IP_REQ_MUX3_DISABLED 

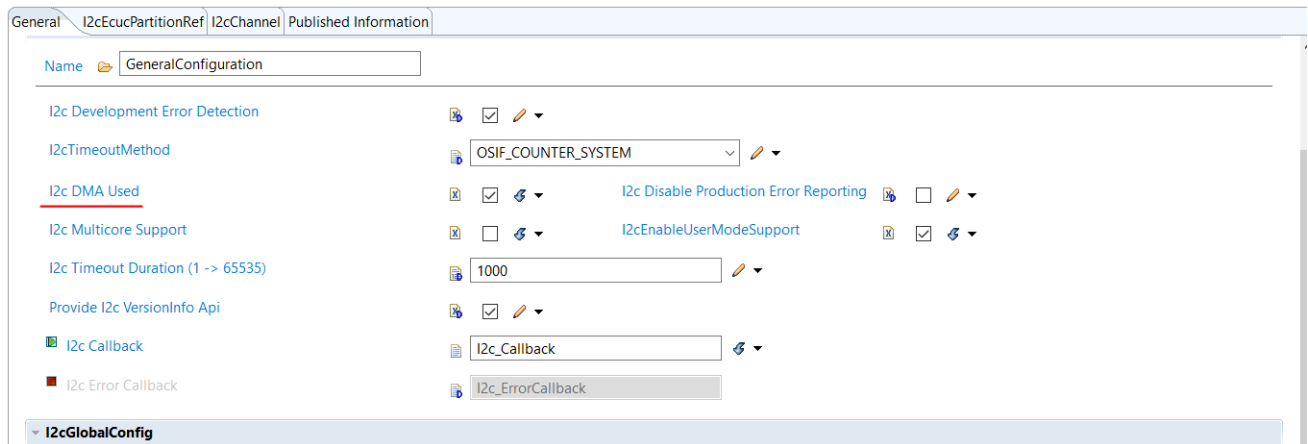
Enable DMA Request  ☒ 

Interrupt

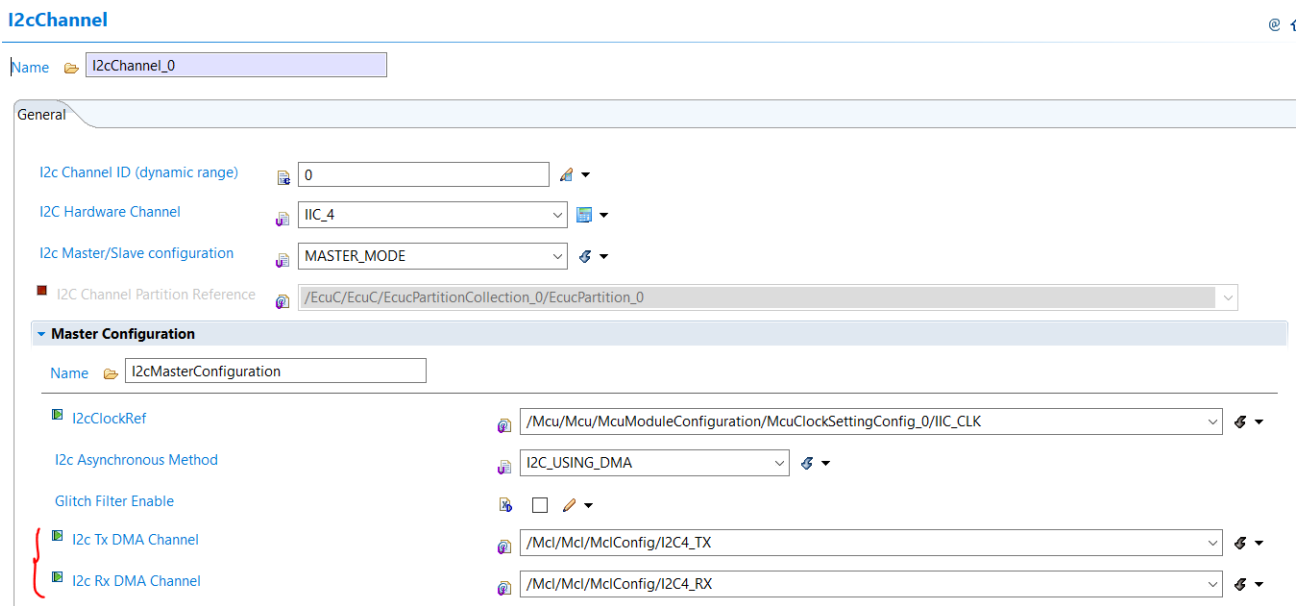
Name  dmaLogicChannelConfig_GlobalInterruptType

Enable Error Interrupt  ☐ 

5. In I2c component, "General" tab, the "I2c DMA Used" should be enabled.



6. For the corresponding i2c channel, select "I2C_USING_DMA" for "I2c Asynchronous Method" field and choose the corresponding Dma channels for "I2c Tx Dma Channel" and "I2c Rx Dma Channel".



3.7 Runtime errors

The driver generates the following DEM errors at runtime.

Function	Error Code	Condition triggering the error
I2c_SyncTransmit	I2C_E_TIMEOUT_FAILURE	Bus is busy when trying to send the slave address for a period of time larger than the configured timeout
I2C_AsyncTransmit	I2C_E_TIMEOUT_FAILURE	Bus is busy when trying to send the slave address for a period of time larger than the configured timeout

3.8 Symbolic Names Disclaimer

All containers having `symbolicNameValue` set to `TRUE` in the AUTOSAR schema will generate defines like:

```
#define <Mip>Conf_<Container_ShortName>_<Container_ID>
```

For this reason it is forbidden to duplicate the names of such containers across the RTD configurations or to use names that may trigger other compile issues (e.g. match existing `#ifdefs` arguments).

Chapter 4

Tresos Configuration Plug-in

This chapter describes the Tresos configuration plug-in for the driver. All the parameters are described below.

- Module [I2c](#)
 - Container [GeneralConfiguration](#)
 - * Parameter [I2cDevErrorDetect](#)
 - * Parameter [I2cTimeoutMethod](#)
 - * Parameter [I2cDmaUsed](#)
 - * Parameter [I2cDmaTransferErrorDetect](#)
 - * Parameter [I2cDisableDemReportErrorStatus](#)
 - * Parameter [I2cMulticoreSupport](#)
 - * Parameter [I2cEnableUserModeSupport](#)
 - * Parameter [I2cTimeoutDuration](#)
 - * Parameter [I2cVersionInfoApi](#)
 - * Parameter [I2c_Callback](#)
 - * Parameter [I2cCallback](#)
 - * Parameter [I2c_ErrorCallback](#)
 - * Parameter [I2cErrorCallback](#)
 - Container [I2cGlobalConfig](#)
 - * Reference [I2cEcucPartitionRef](#)
 - * Container [I2cChannel](#)
 - Parameter [I2cChannelId](#)
 - Parameter [I2cHwChannel](#)
 - Parameter [I2cMasterSlaveConfiguration](#)
 - Reference [I2cChannelEcucPartitionRef](#)
 - Container [I2cMasterConfiguration](#)
 - Parameter [I2cAsyncMethod](#)
 - Parameter [I2cMasterGlitchFilter](#)
 - Parameter [I2cIbcValue](#)
 - Parameter [I2cSCLDivider](#)
 - Parameter [I2cSDAHoldDelay](#)
 - Parameter [I2cSCLHoldStart](#)
 - Parameter [I2cSCLHoldStop](#)

- Parameter [I2cBaudRate](#)
- Reference [I2cClockRef](#)
- Reference [I2cTxDmaChannel](#)
- Reference [I2cRxDmaChannel](#)
- Container [I2cSlaveConfiguration](#)
- Parameter [I2cSlaveAddress](#)
- Parameter [I2cSlaveListening](#)
- Parameter [I2cSlaveGlitchFilter](#)
- * Container [I2cDemEventParameterRefs](#)
 - Reference [I2C_E_TIMEOUT_FAILURE](#)
- Container [CommonPublishedInformation](#)
 - * Parameter [ArReleaseMajorVersion](#)
 - * Parameter [ArReleaseMinorVersion](#)
 - * Parameter [ArReleaseRevisionVersion](#)
 - * Parameter [ModuleId](#)
 - * Parameter [SwMajorVersion](#)
 - * Parameter [SwMinorVersion](#)
 - * Parameter [SwPatchVersion](#)
 - * Parameter [VendorApiInfix](#)
 - * Parameter [VendorId](#)

4.1 Module I2c

Configuration of the Inter-integrated circuit (I2c) module.

Included containers:

- [GeneralConfiguration](#)
- [I2cGlobalConfig](#)
- [CommonPublishedInformation](#)

Property	Value
type	ECUC-MODULE-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantSupport	true
supportedConfigVariants	VARIANT-POST-BUILD, VARIANT-PRE-COMPILE

4.2 Container GeneralConfiguration

GeneralConfiguration

This container contains the global configuration parameters of the Non-Autosar I2c driver.

Note: Implementation Specific Parameter.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.3 Parameter I2cDevErrorDetect

I2cDevErrorDetect

Switches the Development Error Detection and Notification ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	true

4.4 Parameter I2cTimeoutMethod

Configures the timeout method.

Based on this selection a certain timeout method from OsIf will be used in the driver.

Note: If SystemTimer or CustomTimer are selected make sure the corresponding timer is enabled in OsIf General configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	OSIF_COUNTER_DUMMY
literals	['OSIF_COUNTER_DUMMY', 'OSIF_COUNTER_SYSTEM', 'OSIF_COUNTER_CUSTOM']

4.5 Parameter I2cDmaUsed

I2cDmaUsed

Check this in order to be able to use DMA in the I2c driver.

Leaving this unchecked will allow the I2c driver to compile with no dependencies from the Mcl driver.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.6 Parameter I2cDmaTransferErrorDetect

I2cDmaTransferErrorDetect

Switches the Dma transfer error of the i2c module ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.7 Parameter I2cDisableDemReportErrorStatus

I2cDisableDemReportErrorStatus

Switches the Diagnostic Error Reporting and Notification OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.8 Parameter I2cMulticoreSupport

This parameter globally enables the possibility to support multicore.

If I2cMulticoreSupport is disabled, then for all the variants no partition shall be defined.

If I2cMulticoreSupport is enabled, at least one EcucPartition needs to be defined (in all variants).

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.9 Parameter I2cEnableUserModeSupport

When this parameter is enabled, the I2C module will adapt to run from User Mode, with the following measures:

a) Configuring REG_PROT so that the registers under protection can be accessed from user mode

by setting UAA bit in REG_PROT_GCR to 1 (only if applicable)

b) Using 'trusted calls' stubs for all internal function calls that access registers requiring

supervisor mode. (only if applicable)

c) Other module specific measures.

For additional details, please refer to chapter '5.6 User Mode Support' in the IM.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.10 Parameter I2cTimeoutDuration

Specifies the maximum number of loops for blocking function until a timeout is raised in short term wait loops

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	1000
max	65535
min	1

4.11 Parameter I2cVersionInfoApi

I2cVersionInfoApi

Switches the I2c_GetVersionInfo function ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	true

4.12 Parameter I2c_Callback

Enable Callback function

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
default Value	false

4.13 Parameter I2cCallback

I2c general callback. This function will be called for both master and slave events.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	I2c_Callback

4.14 Parameter I2c_ErrorCallback

Enable Callback Error function

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	false

4.15 Parameter I2cErrorCallback

I2c error callback. This function will be called for i2c error events.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-FUNCTION-NAME-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A

Property	Value
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
defaultValue	I2c_ErrorCallback

4.16 Container I2cGlobalConfig

This container contains the global configuration parameter of the I2c driver. This container is a MultipleConfigurationContainer, i.e. this container and its sub-containers exist once per configuration set.

Included subcontainers:

- [I2cChannel](#)
- [I2cDemEventParameterRefs](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.17 Reference I2cEcucPartitionRef

Maps the I2C driver to zero or multiple ECUC partitions to make the modules API available in this partition. The I2C driver will operate as an independent instance in each of the partitions.

Tags: atp.Status=draft

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	Infinite

Property	Value
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/EcuC/EcuPartitionCollection/EcuPartition

4.18 Container I2cChannel

This container contains the configuration (parameters) of the I2c Controller(s).

Note: "User should use unique names for naming the I2c channels across different I2cGlobalConfig Sets."

Included subcontainers:

- [I2cMasterConfiguration](#)
- [I2cSlaveConfiguration](#)

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	Infinite
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE

4.19 Parameter I2cChannelId

Identifies the I2c channel.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	true
lowerMultiplicity	1

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	4
min	0

4.20 Parameter I2cHwChannel

Selects the physical I2c Channel.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	IIC_0
literals	['IIC_0', 'IIC_1', 'IIC_2', 'IIC_3', 'IIC_4']

4.21 Parameter I2cMasterSlaveConfiguration

Selects the master slave configuration.

Select whether the selected channel will be used as Master, Slave or both.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF

Property	Value
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: POST-BUILD
defaultValue	MASTER_MODE
literals	['MASTER_MODE', 'SLAVE_MODE']

4.22 Reference I2cChannelEcucPartitionRef

Maps one single I2C channel to zero or one ECUC partitions.

The ECUC partition referenced is a subset of the ECUC partitions

where the I2C driver is mapped to.

Tags: atp.Status=draft

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	true
multiplicityConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-PRE-COMPILE: PRE-COMPILE
	VARIANT-POST-BUILD: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/EcuC/EcucPartitionCollection/EcucPartition

4.23 Container I2cMasterConfiguration

This container contains the configuration (parameters) of the Master Configuration.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.24 Parameter I2cAsyncMethod

This configures IIC_IBCR[DMAEN].

Configures the asynchronous mechanism used by the 'AsyncTransmit' function (interrupts or DMA).

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-ENUMERATION-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	I2C_USING_INTERRUPTS
literals	['I2C_USING_INTERRUPTS', 'I2C_USING_DMA']

4.25 Parameter I2cMasterGlitchFilter

I2cMasterGlitchFilter

Switches the glitch filter function ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.26 Parameter I2cIbcValue

Configures IIC_IBFD

Configures the I-BUS Clock Rate for I2c Driver.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	0
max	191
min	0

4.27 Parameter I2cSCLDivider

SCL Divider: Calculated from the IBC value and based on the I2C divider and hold values table in the Reference manual.

The calculated SCL Divider value should coincide (be equal to) with $\text{MCU_Bus_Frequency} / \text{SCL_Clock_Frequency}$

In other words, 1 SCL Cycle = 'I2cSCLDivider' Bus Clock Cycles

When multiplied with the bus clock period (i.e. $1 / \text{'I2cClockRef/McuClockReferencePointFrequency'}$), it gives the amount of time to transfer a bit.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	36
max	4294967295
min	0

4.28 Parameter I2cSDAHoldDelay

SDA Hold: Calculated from the IBC value and based on the I2C divider and hold values table in the Reference manual.

When multiplied with the bus clock period (i.e. $1 / \text{'I2cClockRef/McuClockReferencePointFrequency'}$):

It corresponds with the tHD;DAT symbol (data hold time) from the I2c specification.

It gives the amount of time after the falling edge of the SCL that the data will remain valid.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	14
max	4294967295
min	0

4.29 Parameter I2cSCLHoldStart

SCL Hold (start): Calculated from the IBC value and based on the I2C divider and hold values table in the Reference manual.

When multiplied with the bus clock period (i.e. $1 / \text{'I2cClockRef/McuClockReferencePointFrequency'}$):

It corresponds with the tHD;STA symbol (hold time for [repeated] START condition) from the I2c specification.

It gives the delay from the falling edge of SDA to the falling edge of SCL during a (repeated) START condition.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	7
max	4294967295
min	0

4.30 Parameter I2cSCLHoldStop

SCL Hold (stop): Calculated from the IBC value and based on the I2C divider and hold values table in the Reference manual.

When multiplied with the bus clock period (i.e. $1 / \text{'I2cClockRef/McuClockReferencePointFrequency'}$):

It corresponds with the tSU:STO symbol (set-up time for STOP condition) from the I2c specification.

It gives the delay from the rising edge of SCL to the rising edge of SDA during a STOP condition.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false

Property	Value
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	19
max	4294967295
min	0

4.31 Parameter I2cBaudRate

Calculated as $\text{Bus Frequency} / \text{SCL_Divider}$ (i.e. $\text{'I2cClockRef/McuClockReferencePointFrequency'} / \text{I2cSCLDivider}$)

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-FLOAT-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0.0
max	1000000.0
min	0.0

4.32 Reference I2cClockRef

Reference to the I2c clock source configuration, which is set in the MCU driver configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF

Property	Value
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Mcu/McuModuleConfiguration/McuClockSetting↔ Config/McuClockReferencePoint

4.33 Reference I2cTxDmaChannel

Reference to the TX DMA channel, which is set in the Mcl driver configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcuDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.34 Reference I2cRxDmaChannel

Reference to the RX DMA channel, which is set in the Mcl driver configuration.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	False
destination	/AUTOSAR/EcucDefs/Mcl/MclConfig/dmaLogicChannel_Type

4.35 Container I2cSlaveConfiguration

This container contains the Slave configuration (parameters).

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.36 Parameter I2cSlaveAddress

The address of the slave.

This configures IIC_IBAD[7:1].

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP

Property	Value
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	true
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	0
max	127
min	0

4.37 Parameter I2cSlaveListening

I2cSlaveListening

Check this in order to chose slave mode (always listening or on demand only).

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
defaultValue	false

4.38 Parameter I2cSlaveGlitchFilter

I2cMasterGlitchFilter

Switches the glitch filter function ON or OFF.

Note: Implementation Specific Parameter.

Property	Value
type	ECUC-BOOLEAN-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PRE-COMPILE
	VARIANT-PRE-COMPILE: PRE-COMPILE
default Value	false

4.39 Container I2cDemEventParameterRefs

Container for the references to DemEventParameter elements which shall be invoked using the API Dem_SetEventStatus API in case the corresponding error occurs. The EventId is taken from the referenced DemEventParameter's DemEventId value. The standardized errors are provided in the container and can be extended by vendor specific error references.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	0
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE

4.40 Reference I2C_E_TIMEOUT_FAILURE

Reference to the DemEventParameter which shall be issued when the error "Timeout caused by hardware error" has occurred.

Property	Value
type	ECUC-REFERENCE-DEF
origin	NXP
lowerMultiplicity	0

Property	Value
upperMultiplicity	1
postBuildVariantMultiplicity	false
multiplicityConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: POST-BUILD
	VARIANT-PRE-COMPILE: PRE-COMPILE
requiresSymbolicNameValue	true
destination	/AUTOSAR/EcucDefs/Dem/DemConfigSet/DemEventParameter

4.41 Container CommonPublishedInformation

Common container, aggregated by all modules.

It contains published information about vendor and versions.

Included subcontainers:

- None

Property	Value
type	ECUC-PARAM-CONF-CONTAINER-DEF
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

4.42 Parameter ArReleaseMajorVersion

Major version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.43 Parameter ArReleaseMinorVersion

Minor version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.44 Parameter ArReleaseRevisionVersion

Revision version number of AUTOSAR specification on which the appropriate implementation is based on.

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION

Property	Value
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.45 Parameter ModuleId

Module ID of this module from Module List.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	255
max	255
min	255

4.46 Parameter SwMajorVersion

Major version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A

Property	Value
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	4
max	4
min	4

4.47 Parameter SwMinorVersion

Minor version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	0
max	0
min	0

4.48 Parameter SwPatchVersion

Patch level version number of the vendor specific implementation of the module. The numbering is vendor specific.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1

Property	Value
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	2
max	2
min	2

4.49 Parameter VendorApiInfix

In driver modules which can be instantiated several times on a single ECU, BSW00347 requires that the name of APIs is extended by the VendorId and a vendor specific name.

This parameter is used to specify the vendor specific name. In total, the Implementation specific name is generated as follows:

<ModuleName>__>VendorId>__<VendorApiInfix>.

E.g. assuming that the VendorId of the implementor is 123 and the implementer chose a VendorApiInfix of "v11r456" a api name

Can_Write defined in the SWS will translate to Can_123_v11r456Write.

This parameter is mandatory for all modules with upper multiplicity >

1. It shall not be used for modules with upper multiplicity =1.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-STRING-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	

4.50 Parameter VendorId

Vendor ID of the dedicated implementation of this module according to the AUTOSAR vendor list.

Note: Implementation Specific Parameter

Property	Value
type	ECUC-INTEGER-PARAM-DEF
origin	NXP
symbolicNameValue	false
lowerMultiplicity	1
upperMultiplicity	1
postBuildVariantMultiplicity	N/A
multiplicityConfigClasses	N/A
postBuildVariantValue	false
valueConfigClasses	VARIANT-POST-BUILD: PUBLISHED-INFORMATION
	VARIANT-PRE-COMPILE: PUBLISHED-INFORMATION
defaultValue	43
max	43
min	43

This chapter describes the Tresos configuration plug-in for the I2C Driver. The most of the parameters are described below.



Chapter 5

Module Index

5.1 Software Specification

Here is a list of all modules:

I2c Driver	43
IIC Driver	58

Chapter 6

Module Documentation

6.1 I2c Driver

6.1.1 Detailed Description

Data Structures

- struct [I2C_Ipw_HwChannelConfigType](#)
Channel configuration structure for master and slave. [More...](#)
- struct [I2c_Ipw_HwChannelConfigType](#)
The structure contains the hardware channel configuration type. [More...](#)
- struct [I2c_HwUnitConfigType](#)
Structure that contains I2C Hw configuration. [More...](#)
- struct [I2c_DemConfigType](#)
DEM error reporting configuration. [More...](#)
- struct [I2c_ConfigType](#)
This type contains initialization data. [More...](#)
- struct [I2c_RequestType](#)
Definition for Request Buffer. This is the structure which is passed to I2c_SyncTransmit or I2c_AsyncTransmit function. This holds the necessary information required for the communication of I2C Hw with the Slave device. [More...](#)

Macros

- [#define I2C_STOP_SEC_CONFIG_DATA_UNSPECIFIED](#)
Export Post-Build configurations.
- [#define I2C_START_SEC_VAR_INIT_8](#)

Types Reference

- typedef uint8 [I2c_HwUnitType](#)
This gives the numeric ID (hardware number) of an I2C hw Unit.
- typedef uint8 [I2c_PartCoreType](#)
This gives the numeric ID (partition number) of an I2C hw Unit.
- typedef uint16 [I2c_AddressType](#)
Type Address Value of Device and its register value.
- typedef uint8 [I2c_DataType](#)
Type Data to be sent or received.

Enum Reference

- enum [I2c_HwChannelType](#)
Definition of the hardware channel type.
- enum [I2c_ApiFunctionIdType](#)
API functions service IDs.
- enum [I2c_StatusType](#)
Definition for different state and errors of Operation Status.
- enum [I2c_AsynchronousMethodType](#)
Asynchronous method used.
- enum [I2c_MasterSlaveModeType](#)
Definition of the master/slave mode of an I2C hw unit.
- enum [I2c_DataDirectionType](#)
Definition of the type of activation or procession mechanism of an I2C hw unit.

Function Reference

- void [I2c_Init](#) (const [I2c_ConfigType](#) *Config)
Initializes the I2C module.
- void [I2c_DeInit](#) (void)
DeInitializes the I2C module.
- Std_ReturnType [I2c_SyncTransmit](#) (uint8 Channel, const [I2c_RequestType](#) *RequestPtr)
Sends or receives an I2C message blocking.
- Std_ReturnType [I2c_AsyncTransmit](#) (uint8 Channel, const [I2c_RequestType](#) *RequestPtr)
Starts an asynchronous transmission on the I2C bus.
- [I2c_StatusType](#) [I2c_GetStatus](#) (uint8 Channel)
Gets the status of an I2C channel.
- Std_ReturnType [I2c_PrepareSlaveBuffer](#) (uint8 Channel, uint8 NumberOfBytes, [I2c_DataType](#) *DataBuffer)
Prepare the RX or TX buffer for a slave channel.
- Std_ReturnType [I2c_StartListening](#) (uint8 Channel)
Makes a slave channel available for processing requests (addressings).

Variables

- sint8 [I2c_as8ChannelHardwareMap](#) [I2C_HW_MAX_MODULES]
- const [I2c_DemConfigType](#) * [I2c_apDemCfg](#) [I2C_MAX_CORE_ID]

6.1.2 Data Structure Documentation

6.1.2.1 struct IIC_Ipw_HwChannelConfigType

Channel configuration structure for master and slave.

This structure is used to provide configuration parameters for the I2C hw channel at initialization time. Implements :

Definition at line 106 of file [I2c_Ipw_Types.h](#).

6.1.2.2 struct I2c_Ipw_HwChannelConfigType

The structure contains the hardware channel configuration type.

Definition at line 115 of file [I2c_Ipw_Types.h](#).

6.1.2.3 struct I2c_HwUnitConfigType

Structure that contains I2C Hw configuration.

It contains the information specific to one I2C Hw unit

Definition at line 233 of file [I2c_Types.h](#).

Data Fields

- const [I2c_HwUnitType](#) [I2c_HwUnit](#)
Numeric instance value of I2C Hw Unit.
- const [I2c_PartCoreType](#) [I2c_PartitionId](#)
Master/slave mode configuration of the I2C Hw Unit.
- const [I2c_MasterSlaveModeType](#) [MasterSlaveConfig](#)
Hardware channel type.
- const [I2c_HwChannelType](#) [I2c_Ipw_ChannelType](#)
Structure containing the hardware specific configuration for the channel.

6.1.2.3.1 Field Documentation

6.1.2.3.1.1 I2c_HwUnit `const I2c_HwUnitType I2c_HwUnit`

Numeric instance value of I2C Hw Unit.

<

Numeric Partition Id

Definition at line 236 of file I2c_Types.h.

6.1.2.3.1.2 I2c_PartitionId `const I2c_PartCoreType I2c_PartitionId`

Master/slave mode configuration of the I2C Hw Unit.

Definition at line 239 of file I2c_Types.h.

6.1.2.3.1.3 MasterSlaveConfig `const I2c_MasterSlaveModeType MasterSlaveConfig`

Hardware channel type.

Definition at line 242 of file I2c_Types.h.

6.1.2.3.1.4 I2c_Ipw_ChannelType `const I2c_HwChannelType I2c_Ipw_ChannelType`

Structure containing the hardware specific configuration for the channel.

Definition at line 245 of file I2c_Types.h.

6.1.2.4 struct I2c_DemConfigType

DEM error reporting configuration.

This structure contains information DEM error reporting

Definition at line 256 of file I2c_Types.h.

6.1.2.5 struct I2c_ConfigType

This type contains initialization data.

This contains initialization data for the I2C driver. It shall contain:

- The number of I2C modules to be configured
- Dem error reporting configuration
- I2C dependent properties for used HW units

Definition at line 270 of file I2c_Types.h.

Data Fields

- const [I2c_PartCoreType](#) I2c_CoreId
Numeric Partition Id.
- const [I2c_DemConfigType](#) * I2c_DemConfig
Pointer to I2c hardware unit configuration.

6.1.2.5.1 Field Documentation

6.1.2.5.1.1 I2c_CoreId `const I2c_PartCoreType I2c_CoreId`

Numeric Partition Id.

<

DEM error reporting configuration.

Definition at line 273 of file I2c_Types.h.

6.1.2.5.1.2 I2c_DemConfig `const I2c_DemConfigType* I2c_DemConfig`

Pointer to I2c hardware unit configuration.

Definition at line 277 of file I2c_Types.h.

6.1.2.6 struct I2c_RequestType

Definition for Request Buffer. This is the structure which is passed to I2c_SyncTransmit or I2c_AsyncTransmit function. This holds the necessary information required for the communication of I2C Hw with the Slave device.

Definition at line 291 of file I2c_Types.h.

Data Fields

- [I2c__AddressType SlaveAddress](#)
Slave Device Address.
- boolean [BitsSlaveAddressSize](#)
If this is true the data will be sent with high speed enabled (if hardware support exists).
- boolean [HighSpeedMode](#)
When this is true, NACK will be ignored during the address cycle.
- boolean [ExpectNack](#)
When this is true, a repeated start (Sr) will be issued on the bus instead of a STOP at the end of the transfer.
- boolean [RepeatedStart](#)
Buffer Size : The number of bytes for reading or writing.
- uint16 [BufferSize](#)
Direction of the data. Can be either Send or Receive.
- [I2c__DataDirectionType DataDirection](#)
Buffer to Store or to transmit Serial data.

6.1.2.6.1 Field Documentation

6.1.2.6.1.1 SlaveAddress [I2c__AddressType](#) SlaveAddress

Slave Device Address.

<

This is true when the slave address is 10 bits, when false the address is on 7 bits.

Definition at line 294 of file I2c__Types.h.

6.1.2.6.1.2 BitsSlaveAddressSize [boolean](#) BitsSlaveAddressSize

If this is true the data will be sent with high speed enabled (if hardware support exists).

Definition at line 297 of file I2c__Types.h.

6.1.2.6.1.3 HighSpeedMode [boolean](#) HighSpeedMode

When this is true, NACK will be ignored during the address cycle.

Definition at line 300 of file I2c__Types.h.

6.1.2.6.1.4 ExpectNack `boolean ExpectNack`

When this is true, a repeated start (Sr) will be issued on the bus instead of a STOP at the end of the transfer.

Definition at line 303 of file I2c_Types.h.

6.1.2.6.1.5 RepeatedStart `boolean RepeatedStart`

Buffer Size : The number of bytes for reading or writing.

Definition at line 306 of file I2c_Types.h.

6.1.2.6.1.6 BufferSize `uint16 BufferSize`

Direction of the data. Can be either Send or Receive.

Definition at line 309 of file I2c_Types.h.

6.1.2.6.1.7 DataDirection `I2c_DataDirectionType DataDirection`

Buffer to Store or to transmit Serial data.

Definition at line 312 of file I2c_Types.h.

6.1.3 Macro Definition Documentation**6.1.3.1 I2C_STOP_SEC_CONFIG_DATA_UNSPECIFIED**

```
#define I2C_STOP_SEC_CONFIG_DATA_UNSPECIFIED
```

Export Post-Build configurations.

I2c_h_REF_1 MISRA 2012 Required Rule 19.15, Repeated include file I2c_h_REF_3 MISRA 2012 Advisory Rule 19.1, only preprocessor statements and comments before '#include'

Definition at line 252 of file CDD_I2c.h.

6.1.3.2 I2C_START_SEC_VAR_INIT_8

```
#define I2C_START_SEC_VAR_INIT_8
```

I2c_h_REF_1 MISRA 2012 Required Rule 19.15, Repeated include file I2c_h_REF_3 MISRA 2012 Advisory Rule 19.1, only preprocessor statements and comments before '#include'

Definition at line 261 of file CDD_I2c.h.

6.1.4 Types Reference

6.1.4.1 I2c_HwUnitType

```
typedef uint8 I2c_HwUnitType
```

This gives the numeric ID (hardware number) of an I2C hw Unit.

Definition at line 203 of file I2c_Types.h.

6.1.4.2 I2c_PartCoreType

```
typedef uint8 I2c_PartCoreType
```

This gives the numeric ID (partition number) of an I2C hw Unit.

Definition at line 209 of file I2c_Types.h.

6.1.4.3 I2c_AddressType

```
typedef uint16 I2c_AddressType
```

Type Address Value of Device and its register value.

Definition at line 217 of file I2c_Types.h.

6.1.4.4 I2c_DataType

```
typedef uint8 I2c_DataType
```

Type Data to be sent or received.

Definition at line 225 of file I2c_Types.h.

6.1.5 Enum Reference

6.1.5.1 I2c_HwChannelType

```
enum I2c_HwChannelType
```

Definition of the hardware channel type.

Enumerator

I2C_IIC_CHANNEL	This is used for IIC channels.
-----------------	--------------------------------

Definition at line 124 of file I2c_Ipw_Types.h.

6.1.5.2 I2c_ApiFunctionIdType

enum [I2c_ApiFunctionIdType](#)

API functions service IDs.

Service IDs of the I2C API.

Enumerator

I2C_INIT_ID	I2c_Init() ID.
I2C_DEINIT_ID	I2c_DeInit() ID.
I2C_SYNCTRANSMIT_ID	I2c_SyncTransmit() ID.
I2C_ASYNCTRANSMIT_ID	I2c_AsyncTransmit() ID.
I2C_GETSTATUS_ID	I2c_GetStatus() ID.
I2C_PREPARESLAVEBUFFER_ID	I2c_PrepareSlaveBuffer() ID.
I2C_STARTLISTENING_ID	I2c_StartListening() ID.
I2C_GETVERSIONINFO_ID	I2c_GetVersionInfo() ID.

Definition at line 130 of file I2c_Types.h.

6.1.5.3 I2c_StatusType

enum [I2c_StatusType](#)

Definition for different state and errors of Operation Status.

Enumerator

I2C_CH_IDLE	Status Indication I2C channel is idle.
I2C_CH_SEND	Status Indication send operation is ongoing.
I2C_CH_RECEIVE	Status Indication receiving operation is ongoing.
I2C_CH_FINISHED	Status Indication operation is finished.
I2C_CH_ERROR_PRESENT	Status Indication an error is present.

Definition at line 149 of file I2c_Types.h.

6.1.5.4 I2c_AsynchronousMethodType

enum `I2c_AsynchronousMethodType`

Asynchronous method used.

Enumerator

I2C_INTERRUPT_MODE	Asynchronous Mechanism using interrupts.
I2C_DMA_MODE	Asynchronous Mechanism using DMA.

Definition at line 164 of file I2c_Types.h.

6.1.5.5 I2c_MasterSlaveModeType

enum `I2c_MasterSlaveModeType`

Definition of the master/slave mode of an I2C hw unit.

Definition at line 173 of file I2c_Types.h.

6.1.5.6 I2c_DataDirectionType

enum `I2c_DataDirectionType`

Definition of the type of activation or proccession mechanism of an I2C hw unit.

Enumerator

I2C_SEND_DATA	Used to send data to a slave.
I2C_RECEIVE_DATA	Used to receive data from a slave.

Definition at line 187 of file I2c_Types.h.

6.1.6 Function Reference

6.1.6.1 I2c_Init()

```
void I2c_Init (
    const I2c_ConfigType * Config )
```

Initializes the I2C module.

This function performs software initialization of I2C driver:

- Maps logical channels to hardware channels
- Initializes all channels
- Sets driver state machine to I2C_INIT.

Parameters

in	<i>pConfig</i>	Pointer to I2C driver configuration set.
----	----------------	--

Returns

void

Note

Service ID: 0x00.

Synchronous, non re-entrant function.

6.1.6.2 I2c_DeInit()

```
void I2c_DeInit (
    void )
```

DeInitializes the I2C module.

This function performs software de initialization of I2C modules to reset values. The service influences only the peripherals, which are allocated by static configuration and the runtime configuration set passed by the previous call of [I2c_Init\(\)](#). The driver needs to be initialized before calling [I2c_DeInit\(\)](#). Otherwise, the function I2c_DeInit shall raise the development error I2C_E_UNINIT and leave the desired de initialization functionality without any action.

Parameters

in	<i>void</i>	
----	-------------	--

Returns

void

Note

Service ID: 0x01.
Synchronous, non re-entrant function.

6.1.6.3 I2c__SyncTransmit()

```
Std_ReturnType I2c_SyncTransmit (
    uint8 Channel,
    const I2c_RequestType * RequestPtr )
```

Sends or receives an I2C message blocking.

Sends the slave address and based on the direction of the message it sends or receives data by using a blocking mechanism.

Parameters

in	<i>u8Channel</i>	I2C channel to be addressed.
in	<i>pRequestPtr</i>	Pointer to data information to be used

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the I2C Channel is not valid or I2C driver is not initialized or pRequestPtr is NULL or I2C Channel is in busy state.
<i>E_OK</i>	Otherwise.

Note

Service ID: 0x02.
Synchronous, non reentrant function.

6.1.6.4 I2c_AsyncTransmit()

```
Std_ReturnType I2c_AsyncTransmit (
    uint8 Channel,
    const I2c_RequestType * RequestPtr )
```

Starts an asynchronous transmission on the I2C bus.

Sends the slave address and enables the interrupts that will send or receive data depending on the direction of the message.

Parameters

in	<i>u8Channel</i>	I2C channel to be addressed.
in	<i>pRequestPtr</i>	Pointer to data information to be used

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the I2C Channel is not valid or I2C driver is not initialized or pRequestPtr is NULL or I2C Channel is in busy state.
<i>E_OK</i>	Otherwise.

Note

Service ID: 0x03.

Synchronous, non reentrant function.

6.1.6.5 I2c_GetStatus()

```
I2c_StatusType I2c_GetStatus (
    uint8 Channel )
```

Gets the status of an I2C channel.

Gets the status of an I2C channel and checks for errors.

Parameters

in	<i>u8Channel</i>	I2C channel to be addressed.
----	------------------	------------------------------

Module Documentation

Returns

I2C_StatusType.

Return values

<i>I2C_CH_IDLE</i>	If the I2C Channel is in default state.
<i>I2C_CH_SEND</i>	If the I2C Channel is busy sending data.
<i>I2C_CH_RECEIVE</i>	If the I2C Channel is busy receiving data.
<i>I2C_CH_FINISHED</i>	If the I2C Channel finished the last transmission (sending or receiving data) successfully with no errors.
<i>I2C_CH_ERROR_PRESENT</i>	If the I2C Channel encountered an error during the last transmission.

Note

Service ID: 0x04.

Synchronous, non re-entrant function.

6.1.6.6 I2c_PrepareSlaveBuffer()

```
Std_ReturnType I2c_PrepareSlaveBuffer (  
    uint8 Channel,  
    uint8 NumberOfBytes,  
    I2c_DataType * DataBuffer )
```

Prepare the RX or TX buffer for a slave channel.

Prepares a RX or TX buffer that will be used to receive data or send data when requested by the master.

Parameters

in	<i>Channel</i>	I2C channel to be addressed.
in	<i>NumberOfBytes</i>	Maximum number of bytes to be sent or received.
in	<i>DataBuffer</i>	Pointer to data buffer

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the I2C Channel is not valid or I2C driver is not initialized or DataBuffer is NULL or I2C Channel is a master channel.
<i>E_OK</i>	Otherwise.

Note

Service ID: 0x05.

Synchronous, non reentrant function.

6.1.6.7 I2c_StartListening()

```
Std_ReturnType I2c_StartListening (
    uint8 Channel )
```

Makes a slave channel available for processing requests (addressings).

When called, the slave channel becomes available for starting incoming or outgoing transfers.

Parameters

in	<i>u8Channel</i>	I2C channel to be addressed.
----	------------------	------------------------------

Returns

Std_ReturnType.

Return values

<i>E_NOT_OK</i>	If the I2C Channel is not valid or I2C driver is not initialized or I2C Channel is a master channel.
<i>E_OK</i>	Otherwise.

Note

Service ID: 0x06.

Synchronous, non reentrant function.

6.1.7 Variable Documentation

6.1.7.1 I2c_as8ChannelHardwareMap

```
sint8 I2c_as8ChannelHardwareMap[I2C_HW_MAX_MODULES] [extern]
```

I2c_h_REF_1 MISRA 2012 Required Rule 19.15, Repeated include file I2c_h_REF_3 MISRA 2012 Advisory Rule 19.1, only preprocessor statements and comments before '#include' I2c_h_REF_4 This is incorrectly reported by the PCLint tool.

6.1.7.2 I2c_apDemCfg

```
const I2c_DemConfigType* I2c_apDemCfg[I2C_MAX_CORE_ID] [extern]
```

I2c_h_REF_4 This is incorrectly reported by the PCLint tool.

6.2 IIC Driver

6.2.1 Detailed Description

Data Structures

- struct [I2c_Ip_SlaveStateType](#)
Slave internal context structure. [More...](#)
- struct [I2c_Ip_SlaveConfigType](#)
Slave configuration structure. [More...](#)
- struct [I2c_Ip_MasterStateType](#)
Master internal context structure. [More...](#)
- struct [I2c_Ip_MasterConfigType](#)
Master configuration structure. [More...](#)

Macros

- `#define` [I2C_START_SEC_CODE](#)
- `#define` [I2c_Ip_MasterInit](#)(Instance, UserConfigPtr)
- `#define` [I2c_Ip_SlaveInit](#)(Instance, UserConfigPtr)
- `#define` [I2c_Ip_MasterDeinit](#)(Instance)
- `#define` [I2c_Ip_SlaveDeinit](#)(Instance)
- `#define` [I2c_Ip_MasterSendData](#)(Instance, TxBuff, TxSize, SendStop)
- `#define` [I2c_Ip_MasterReceiveDataBlocking](#)(Instance, RxBuff, RxSize, SendStop, Timeout)
- `#define` [I2c_Ip_MasterSendDataBlocking](#)(Instance, TxBuff, TxSize, SendStop, Timeout)
- `#define` [I2c_Ip_MasterReceiveData](#)(Instance, RxBuff, RxSize, SendStop)
- `#define` [I2c_Ip_MasterSetBaudRate](#)(Instance, PrescaledShift, PrescalerDivider, ShiftTapPoint)
- `#define` [I2c_Ip_ModuleIRQHandler](#)(Instance)

Enum Reference

- enum [I2c_Ip_SlaveEventType](#)
Define the enum of the events which can trigger I2C slave callback.
- enum [I2c_Ip_MasterEventType](#)
Define the enum of the events which can trigger I2C master callback.
- enum [I2c_Ip_ModuleType](#)
I2C module types Implements : `i2c_module_type_Class`.
- enum [I2c_Ip_StatusType](#)
- enum [I2c_Ip_TransferType](#)
Type of I2C transfer (based on interrupts or DMA). implements `I2c_Ip_TransferType_enum`.

Function Reference

- [I2c_Ip_StatusType I2c_Ip_MasterInit_Privileged](#) (uint8 Instance, const [I2c_Ip_MasterConfigType](#) *User←ConfigPtr)
Initialize the I2C master mode driver.
- [I2c_Ip_StatusType I2c_Ip_SlaveInit_Privileged](#) (uint8 Instance, const [I2c_Ip_SlaveConfigType](#) *User←ConfigPtr)
Initialize the I2C slave mode driver.
- [I2c_Ip_StatusType I2c_Ip_MasterGetTransferStatus](#) (uint8 Instance)
Return the current status of the I2C master transfer.
- [I2c_Ip_StatusType I2c_Ip_SlaveGetTransferStatus](#) (uint8 Instance)
Return the current status of the I2C slave transfer.
- [I2c_Ip_StatusType I2c_Ip_MasterDeinit_Privileged](#) (uint8 Instance)
De-initialize the I2C master mode driver.
- [I2c_Ip_StatusType I2c_Ip_SlaveDeinit_Privileged](#) (uint8 Instance)
Slave module de-initialization function This function de-initializes the I2C driver in slave mode. The driver can't be used again until reinitialized. The context structure is no longer needed by the driver and can be freed after calling this function.
- [I2c_Ip_StatusType I2c_Ip_SlaveSetBuffer](#) (uint8 Instance, uint8 *TxBuff, uint32 TxSize)
Provide a buffer for transmitting data.
- void [I2c_Ip_StartListening](#) (uint8 Instance)
Makes a slave channel available for processing requests (addressing) in case slave is in not listening mode.
- [I2c_Ip_StatusType I2c_Ip_MasterSendData_Privileged](#) (uint8 Instance, const uint8 *TxBuff, uint32 TxSize, boolean SendStop)
Perform a non-blocking send transaction on the I2C bus.
- [I2c_Ip_StatusType I2c_Ip_MasterReceiveDataBlocking_Privileged](#) (uint8 Instance, uint8 *RxBuff, uint32 RxSize, boolean SendStop, uint32 Timeout)
Perform a blocking receive transaction on the I2C bus.
- [I2c_Ip_StatusType I2c_Ip_MasterSendDataBlocking_Privileged](#) (uint8 Instance, const uint8 *TxBuff, uint32 TxSize, boolean SendStop, uint32 Timeout)
Perform a blocking send transaction on the I2C bus.
- void [I2c_Ip_MasterSetSlaveAddress](#) (uint8 Instance, uint8 Address)
Set the slave address for the I2C communication.
- [I2c_Ip_StatusType I2c_Ip_MasterReceiveData_Privileged](#) (uint8 Instance, uint8 *RxBuff, uint32 RxSize, boolean SendStop)
Perform a non-blocking receive transaction on the I2C bus.
- [I2c_Ip_StatusType I2c_Ip_MasterGetBaudRate](#) (uint8 Instance, uint32 InputClock, uint32 *BaudRate)
Get the master baud rate for the I2C communication.
- [I2c_Ip_StatusType I2c_Ip_MasterSetBaudRate_Privileged](#) (uint8 Instance, uint8 PrescaledShift, uint8 PrescalerDivider, uint8 ShiftTapPoint)
Set the master baud rate for the I2C communication.
- void [I2c_Ip_ModuleIRQHandler_Privileged](#) (uint8 Instance)
Interrupt handler signature for I2C module.
- void [I2c_Ip_SetMasterCallback](#) (uint8 Instance, [I2c_Ip_MasterCallbackType](#) MasterCallback)
Set the master callback.
- void [I2c_Ip_SetSlaveCallback](#) (uint8 Instance, [I2c_Ip_SlaveCallbackType](#) SlaveCallback)
Set the slave callback.

6.2.2 Data Structure Documentation

6.2.2.1 struct I2c_Ip_SlaveStateType

Slave internal context structure.

This structure is used by the slave-mode driver for its internal logic. It must be provided by the application through the I2C_DRV_SlaveInit() function. The application should make no assumptions about the content of this structure.

Implements: i2c_slave_state_t_Class

Definition at line 154 of file I2c_Ip_Types.h.

6.2.2.2 struct I2c_Ip_SlaveConfigType

Slave configuration structure.

This structure is used to provide configuration parameters for the I2C slave at initialization time. implements I2c_Ip_SlaveConfigType_struct

Definition at line 178 of file I2c_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	SlaveAddress	7 bit slave address
boolean	ListeningMode	
boolean	GlitchFilter	Specifies if glitch filter is enabled
I2c_Ip_SlaveCallbackType	SlaveCallback	Slave callback function
uint8	CallbackParam	Parameter for the slave callback function
uint8	SlaveIndex	Slave Index

6.2.2.3 struct I2c_Ip_MasterStateType

Master internal context structure.

This structure is used by the master-mode driver for its internal logic. It must be provided by the application through the I2C_DRV_MasterInit() function. The application should make no assumptions about the content of this structure.

Definition at line 195 of file I2c_Ip_Types.h.

6.2.2.4 struct I2c_Ip_MasterConfigType

Master configuration structure.

This structure is used to provide configuration parameters for the I2C master at initialization time. implements I2c_Ip_MasterConfigType_struct

Definition at line 225 of file I2c_Ip_Types.h.

Data Fields

Type	Name	Description
uint8	SlaveAddress	Slave address, 7-bit
uint8	BaudRate	Value of IBFD register
I2c_Ip_TransferType	TransferType	Type of I2C transfer
uint8	DmaTxChannel	Dma logic Tx channel number. If DMA mode isn't used this field will be ignored
uint8	DmaRxChannel	Dma logic Rx channel number. If DMA mode isn't used this field will be ignored
boolean	GlitchFilter	Specifies if glitch filter is enabled
I2c_Ip_MasterCallbackType	MasterCallback	Master callback function. Note that this function will be called from the interrupt service routine at the end of a transfer, so its execution time should be as small as possible. It can be NULL if you want to check manually the status of the transfer.
uint8	CallbackParam	Parameter for the master callback function
uint8	MasterIndex	Master Index

6.2.3 Macro Definition Documentation

6.2.3.1 I2C_START_SEC_CODE

```
#define I2C_START_SEC_CODE
```

Note

put all I2C code into defined section

Definition at line 159 of file I2c_Ip.h.

6.2.3.2 I2c_Ip_MasterInit

```
#define I2c_Ip_MasterInit(  
    Instance,  
    UserConfigPtr )
```

I2c_Ip_TrustedMacros_h_REF_6 A function should be used in preference to a function-like macro where they are interchangeable.

Definition at line 109 of file I2c_Ip_TrustedMacros.h.

6.2.3.3 I2c_Ip_SlaveInit

```
#define I2c_Ip_SlaveInit(  
    Instance,  
    UserConfigPtr )
```

I2c_Ip_TrustedMacros_h_REF_6 A function should be used in preference to a function-like macro where they are interchangeable.

Definition at line 126 of file I2c_Ip_TrustedMacros.h.

6.2.3.4 I2c_Ip_MasterDeinit

```
#define I2c_Ip_MasterDeinit(  
    Instance )
```

I2c_Ip_TrustedMacros_h_REF_6 A function should be used in preference to a function-like macro where they are interchangeable.

Definition at line 143 of file I2c_Ip_TrustedMacros.h.

6.2.3.5 I2c_Ip_SlaveDeinit

```
#define I2c_Ip_SlaveDeinit(  
    Instance )
```

I2c_Ip_TrustedMacros_h_REF_6 A function should be used in preference to a function-like macro where they are interchangeable.

Definition at line 160 of file I2c_Ip_TrustedMacros.h.

6.2.3.6 I2c_Ip_MasterSendData

```
#define I2c_Ip_MasterSendData(
    Instance,
    TxBuff,
    TxSize,
    SendStop )
```

I2c_Ip_TrustedMacros_h_REF_6 A function should be used in preference to a function-like macro where they are interchangeable.

Definition at line 177 of file I2c_Ip_TrustedMacros.h.

6.2.3.7 I2c_Ip_MasterReceiveDataBlocking

```
#define I2c_Ip_MasterReceiveDataBlocking(
    Instance,
    RxBuff,
    RxSize,
    SendStop,
    Timeout )
```

I2c_Ip_TrustedMacros_h_REF_6 A function should be used in preference to a function-like macro where they are interchangeable.

Definition at line 195 of file I2c_Ip_TrustedMacros.h.

6.2.3.8 I2c_Ip_MasterSendDataBlocking

```
#define I2c_Ip_MasterSendDataBlocking(
    Instance,
    TxBuff,
    TxSize,
    SendStop,
    Timeout )
```

I2c_Ip_TrustedMacros_h_REF_6 A function should be used in preference to a function-like macro where they are interchangeable.

Definition at line 213 of file I2c_Ip_TrustedMacros.h.

6.2.3.9 I2c_Ip_MasterReceiveData

```
#define I2c_Ip_MasterReceiveData(  
    Instance,  
    RxBuff,  
    RxSize,  
    SendStop )
```

I2c_Ip_TrustedMacros_h_REF_6 A function should be used in preference to a function-like macro where they are interchangeable.

Definition at line 230 of file I2c_Ip_TrustedMacros.h.

6.2.3.10 I2c_Ip_MasterSetBaudRate

```
#define I2c_Ip_MasterSetBaudRate(  
    Instance,  
    PrescaledShift,  
    PrescalerDivider,  
    ShiftTapPoint )
```

I2c_Ip_TrustedMacros_h_REF_6 A function should be used in preference to a function-like macro where they are interchangeable.

Definition at line 248 of file I2c_Ip_TrustedMacros.h.

6.2.3.11 I2c_Ip_ModuleIRQHandler

```
#define I2c_Ip_ModuleIRQHandler(  
    Instance )
```

I2c_Ip_TrustedMacros_h_REF_6 A function should be used in preference to a function-like macro where they are interchangeable.

Definition at line 265 of file I2c_Ip_TrustedMacros.h.

6.2.4 Enum Reference

6.2.4.1 I2c_Ip_SlaveEventType

```
enum I2c_Ip_SlaveEventType
```

Define the enum of the events which can trigger I2C slave callback.

This enum should include the events for all platforms implements I2c_Ip_SlaveEventType_enum

Definition at line 70 of file I2c_Ip_Callbacks.h.

6.2.4.2 I2c_Ip_MasterEventType

enum `I2c_Ip_MasterEventType`

Define the enum of the events which can trigger I2C master callback.

This enum should include the events for all platforms implements `I2c_Ip_MasterEventType_enum`

Definition at line 88 of file `I2c_Ip_Callbacks.h`.

6.2.4.3 I2c_Ip_ModuleType

enum `I2c_Ip_ModuleType`

I2C module types Implements : `i2c_module_type_Class`.

I2C - Register Layout Typedef

Enumerator

<code>I2C_SLAVE</code>	The I2C module is in slave mode >
<code>I2C_MASTER</code>	The I2C module is in master mode >

Definition at line 105 of file `I2c_Ip_HwAccess.h`.

6.2.4.4 I2c_Ip_StatusType

enum `I2c_Ip_StatusType`

Enumerator

<code>I2C_IP_SUCCESS_STATUS</code>	Status success
<code>I2C_IP_ERROR_STATUS</code>	Error occurred, transfer was not successful
<code>I2C_IP_BUSY_STATUS</code>	I2c module is busy, transfer is in progress
<code>I2C_IP_TIMEOUT_STATUS</code>	Timeout occurred
<code>I2C_IP_RECEIVED_NACK_STATUS</code>	NACK signal received
<code>I2C_IP_TX_UNDERRUN_STATUS</code>	TX underrun error
<code>I2C_IP_RX_OVERRUN_STATUS</code>	RX overrun error
<code>I2C_IP_ARBITRATION_LOST_STATUS</code>	Arbitration lost
<code>I2C_IP_BUS_BUSY_STATUS</code>	I2C bus is busy, cannot start transfer

Definition at line 114 of file I2c_Ip_Types.h.

6.2.4.5 I2c_Ip_TransferType

```
enum I2c_Ip_TransferType
```

Type of I2C transfer (based on interrupts or DMA). implements I2c_Ip_TransferType_enum.

Enumerator

I2C_USING_DMA	The driver will use DMA to perform I2C transfer
I2C_USING_INTERRUPTS	The driver will use interrupts to perform I2C transfer

Definition at line 135 of file I2c_Ip_Types.h.

6.2.5 Function Reference

6.2.5.1 I2c_Ip_MasterInit_Privileged()

```
I2c_Ip_StatusType I2c_Ip_MasterInit_Privileged (
    uint8 Instance,
    const I2c_Ip_MasterConfigType * UserConfigPtr )
```

Initialize the I2C master mode driver.

This function initializes the I2C driver in master mode.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>userConfigPtr</i>	Pointer to the I2C master user configuration structure. The function reads configuration data from this structure and initializes the driver accordingly. The application may free this structure after the function returns.

Returns

Error or success status returned by API implements I2c_Ip_MasterInit_Activity

6.2.5.2 I2c_Ip_SlaveInit_Privileged()

```
I2c_Ip_StatusType I2c_Ip_SlaveInit_Privileged (
    uint8 Instance,
    const I2c_Ip_SlaveConfigType * UserConfigPtr )
```

Initialize the I2C slave mode driver.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>userConfigPtr</i>	Pointer to the I2C slave user configuration structure. The function reads configuration data from this structure and initializes the driver accordingly. The application may free this structure after the function returns.

Returns

Error or success status returned by API implements I2c_Ip_SlaveInit_Activity

6.2.5.3 I2c_Ip_MasterGetTransferStatus()

```
I2c_Ip_StatusType I2c_Ip_MasterGetTransferStatus (
    uint8 Instance )
```

Return the current status of the I2C master transfer.

This function can be called during a non-blocking transmission to check the status of the transfer.

Parameters

<i>Instance</i>	I2C peripheral instance number
-----------------	--------------------------------

Returns

Error or success status returned by API implements I2c_Ip_MasterGetTransferStatus_Activity

6.2.5.4 I2c_Ip_SlaveGetTransferStatus()

```
I2c_Ip_StatusType I2c_Ip_SlaveGetTransferStatus (
    uint8 Instance )
```

Return the current status of the I2C slave transfer.

This function can be called during a non-blocking transmission to check the status of the transfer.

Parameters

<i>Instance</i>	I2C peripheral instance number
-----------------	--------------------------------

Returns

Error or success status returned by API implements I2c_Ip_SlaveGetTransferStatus_Activity

6.2.5.5 I2c_Ip_MasterDeinit_Privileged()

```
I2c_Ip_StatusType I2c_Ip_MasterDeinit_Privileged (
    uint8 Instance )
```

De-initialize the I2C master mode driver.

This function de-initializes the I2C driver in master mode. The driver can't be used again until reinitialized. The context structure is no longer needed by the driver and can be freed after calling this function.

Parameters

<i>Instance</i>	I2C peripheral instance number
-----------------	--------------------------------

Returns

Error or success status returned by API implements I2c_Ip_MasterDeinit_Activity

6.2.5.6 I2c_Ip_SlaveDeinit_Privileged()

```
I2c_Ip_StatusType I2c_Ip_SlaveDeinit_Privileged (
    uint8 Instance )
```

Slave module de-initialization function This function de-initializes the I2C driver in slave mode. The driver can't be used again until reinitialized. The context structure is no longer needed by the driver and can be freed after calling this function.

Parameters

<i>Instance</i>	I2C peripheral instance number
-----------------	--------------------------------

Returns

Error or success status returned by API implements I2c_Ip_SlaveDeinit_Activity

6.2.5.7 I2c_Ip_SlaveSetBuffer()

```
I2c_Ip_StatusType I2c_Ip_SlaveSetBuffer (
    uint8 Instance,
    uint8 * TxBuff,
    uint32 TxSize )
```

Provide a buffer for transmitting data.

This function provides a buffer from which the I2C slave-mode driver can transmit data. It can be called for example from the user callback provided at initialization time, when the driver reports events I2C_SLAVE_EVENT_TX_REQ or I2C_SLAVE_EVENT_TX_EMPTY.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>TxBuff</i>	pointer to the data to be transferred
<i>TxSize</i>	length in bytes of the data to be transferred

Returns

Error or success status returned by API implements I2c_Ip_SlaveSetBuffer_Activity

6.2.5.8 I2c_Ip_StartListening()

```
void I2c_Ip_StartListening (
    uint8 Instance )
```

Makes a slave channel available for processing requests (addressing) in case slave is in not listening mode.

When called, the slave channel becomes available for starting incoming or outgoing transfers.

Parameters

in	<i>Instance</i>	I2C peripheral instance number implements I2c_Ip_StartListening_Activity
----	-----------------	--

6.2.5.9 I2c_Ip_MasterSendData_Privileged()

```
I2c_Ip_StatusType I2c_Ip_MasterSendData_Privileged (
    uint8 Instance,
    const uint8 * TxBuff,
```



```
uint32 TxSize,
boolean SendStop )
```

Perform a non-blocking send transaction on the I2C bus.

This function starts the transmission of a block of data to the currently configured slave address and returns immediately. The rest of the transmission is handled by the interrupt service routine.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>TxBuff</i>	pointer to the data to be transferred
<i>TxSize</i>	length in bytes of the data to be transferred
<i>bSendStop</i>	specifies whether or not to generate stop condition after the transmission

Returns

Error or success status returned by API implements I2c_Ip_MasterSendData_Activity

6.2.5.10 I2c_Ip_MasterReceiveDataBlocking_Privileged()

```
I2c_Ip_StatusType I2c_Ip_MasterReceiveDataBlocking_Privileged (
    uint8 Instance,
    uint8 * RxBuff,
    uint32 RxSize,
    boolean SendStop,
    uint32 Timeout )
```

Perform a blocking receive transaction on the I2C bus.

This function receives a block of data from the currently configured slave address, and only returns when the transmission is complete.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>RxBuff</i>	pointer to the buffer where to store received data
<i>RxSize</i>	length in bytes of the data to be transferred
<i>SendStop</i>	specifies whether or not to generate stop condition after the reception
<i>Timeout</i>	timeout for the transfer in milliseconds

Returns

Error or success status returned by API implements I2c_Ip_MasterReceiveDataBlocking_Activity

6.2.5.11 I2c_Ip_MasterSendDataBlocking_Privileged()

```
I2c_Ip_StatusType I2c_Ip_MasterSendDataBlocking_Privileged (
    uint8 Instance,
    const uint8 * TxBuff,
    uint32 TxSize,
    boolean SendStop,
    uint32 Timeout )
```

Perform a blocking send transaction on the I2C bus.

This function sends a block of data to the currently configured slave address, and only returns when the transmission is complete.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>TxBuff</i>	pointer to the data to be transferred
<i>TxSize</i>	length in bytes of the data to be transferred
<i>SendStop</i>	specifies whether or not to generate stop condition after the transmission
<i>Timeout</i>	timeout for the transfer in milliseconds

Returns

Error or success status returned by API implements I2c_Ip_MasterSendDataBlocking_Activity

6.2.5.12 I2c_Ip_MasterSetSlaveAddress()

```
void I2c_Ip_MasterSetSlaveAddress (
    uint8 Instance,
    uint8 Address )
```

Set the slave address for the I2C communication.

This function sets the slave address which will be used for any future transfer initiated by the I2C master.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>Address</i>	slave 7-bit address implements I2c_Ip_MasterSetSlaveAddress_Activity

6.2.5.13 I2c_Ip_MasterReceiveData_Privileged()

```
I2c_Ip_StatusType I2c_Ip_MasterReceiveData_Privileged (
    uint8 Instance,
    uint8 * RxBuff,
    uint32 RxSize,
    boolean SendStop )
```

Perform a non-blocking receive transaction on the I2C bus.

This function starts the reception of a block of data from the currently configured slave address and returns immediately. The rest of the reception is handled by the interrupt service routine.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>RxBuff</i>	pointer to the buffer where to store received data
<i>RxSize</i>	length in bytes of the data to be transferred
<i>SendStop</i>	specifies whether or not to generate stop condition after the reception

Returns

Error or success status returned by API implements I2c_Ip_MasterReceiveData_Activity

6.2.5.14 I2c_Ip_MasterGetBaudRate()

```
I2c_Ip_StatusType I2c_Ip_MasterGetBaudRate (
    uint8 Instance,
    uint32 InputClock,
    uint32 * BaudRate )
```

Get the master baud rate for the I2C communication.

This function returns the master baud rate of the I2C master module.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>InputClock</i>	I2C input clock in Hz
<i>BaudRate</i>	calculated baud rate in Hz

Returns

success status returned by the API implements I2c_Ip_MasterGetBaudRate_Activity

6.2.5.15 I2c_Ip_MasterSetBaudRate_Privileged()

```
I2c_Ip_StatusType I2c_Ip_MasterSetBaudRate_Privileged (
    uint8 Instance,
    uint8 PrescaledShift,
    uint8 PrescalerDivider,
    uint8 ShiftTapPoint )
```

Set the master baud rate for the I2C communication.

This function sets the master baud rate of the I2C master module.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>PrescaledShift</i>	prescaled shift (mul) value
<i>PrescalerDivider</i>	prescaler divider value
<i>ShiftTapPoint</i>	shift tap to point value implements I2c_Ip_MasterSetBaudRate_Activity

6.2.5.16 I2c_Ip_ModuleIRQHandler_Privileged()

```
void I2c_Ip_ModuleIRQHandler_Privileged (
    uint8 Instance )
```

Interrupt handler signature for I2C module.

Returns

void

6.2.5.17 I2c_Ip_SetMasterCallback()

```
void I2c_Ip_SetMasterCallback (
    uint8 Instance,
    I2c_Ip_MasterCallbackType MasterCallback )
```

Set the master callback.

This function sets the master callback of the I2C master module.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>MasterCallback</i>	master callback implements I2c_Ip_SetMasterCallback_Activity

6.2.5.18 I2c_Ip_SetSlaveCallback()

```
void I2c_Ip_SetSlaveCallback (
    uint8 Instance,
    I2c_Ip_SlaveCallbackType SlaveCallback )
```

Set the slave callback.

This function sets the slave callback of the I2C slave module.

Parameters

<i>Instance</i>	I2C peripheral instance number
<i>SlaveCallback</i>	slave callback implements I2c_Ip_SetSlaveCallback_Activity

How to Reach Us:

Home Page:

nxp.com

Web Support:

nxp.com/support

Information in this document is provided solely to enable system and software implementers to use NXP products. There are no express or implied copyright licenses granted hereunder to design or fabricate any integrated circuits based on the information in this document. NXP reserves the right to make changes without further notice to any products herein.

NXP makes no warranty, representation, or guarantee regarding the suitability of its products for any particular purpose, nor does NXP assume any liability arising out of the application or use of any product or circuit, and specifically disclaims any and all liability, including without limitation consequential or incidental damages. "Typical" parameters that may be provided in NXP data sheets and/or specifications can and do vary in different applications, and actual performance may vary over time. All operating parameters, including "typicals," must be validated for each customer application by customer's technical experts. NXP does not convey any license under its patent rights nor the rights of others. NXP sells products pursuant to standard terms and conditions of sale, which can be found at the following address: nxp.com/SalesTermsandConditions.

NXP, the NXP logo, NXP SECURE CONNECTIONS FOR A SMARTER WORLD, COOLFLUX, EMBRACE, GREENCHIP, HITAG, I2C BUS, ICODE, JCOP, LIFE VIBES, MIFARE, MIFARE CLASSIC, MIFARE DESFire, MIFARE PLUS, MIFARE FLEX, MANTIS, MIFARE ULTRALIGHT, MIFARE4MOBILE, MIGLO, NTAG, ROADLINK, SMARTLX, SMARTMX, STARPLUG, TOPFET, TRENCHMOS, UCODE, Freescale, the Freescale logo, Altivec, C-5, CodeTEST, CodeWarrior, ColdFire, ColdFire+, C-Ware, the Energy Efficient Solutions logo, Kinetis, Layerscape, MagniV, mobileGT, PEG, PowerQUICC, Processor Expert, QorIQ, QorIQ Qonverge, Ready Play, SafeAssure, the SafeAssure logo, StarCore, Symphony, VortiQa, Vybrid, Airfast, BeeKit, BeeStack, CoreNet, Flexis, MXC, Platform in a Package, QUICC Engine, SMARTMOS, Tower, TurboLink, and UMEMS are trademarks of NXP B.V. All other product or service names are the property of their respective owners. ARM, AMBA, ARM Powered, Artisan, Cortex, Jazelle, Keil, SecurCore, Thumb, TrustZone, and Vision are registered trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. ARM7, ARM9, ARM11, big.LITTLE, CoreLink, CoreSight, DesignStart, Mali, mbed, NEON, POP, Sensinode, Socrates, ULINK and Versatile are trademarks of ARM Limited (or its subsidiaries) in the EU and/or elsewhere. All rights reserved. Oracle and Java are registered trademarks of Oracle and/or its affiliates. The Power Architecture and Power.org word marks and the Power and Power.org logos and related marks are trademarks and service marks licensed by Power.org.

© 2023 NXP B.V.

